

Services & HttpClient

A Service is where the app's real work happens; **HttpClient** is Angular's built-in tool for talking to a backend from inside one. Together they keep components focused on one job: rendering the UI.

1 · WHY SERVICES EXIST

Separation of concerns — the restaurant analogy

The **Waiter** (Component): takes your order, brings food to your table, never cooks anything. The **Kitchen** (Service): prepares the actual food — the real work happens here. A Service lets many "waiters" (components) use the same "kitchen" (logic) without duplicating it.

COMPONENT'S JOB	SERVICE'S JOB
Render the UI, handle user events	Call APIs / talk to the backend
Show loading / success / error states	Business logic, calculations, caching
Delegate real work to a Service	Share data & state between components

2 · @INJECTABLE AND THE SINGLETON

providedIn: 'root' — one shared instance

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
}
```

A root-provided Service is created **once**, the first time something injects it — then that same instance is reused everywhere. Perfect for shared state: every consumer sees the same data. Bonus: tree-shakable — if nothing ever injects it, it isn't included in the final bundle.

3 · HTTPCLIENT BASICS

A typed GET request, end to end

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
  private baseUrl = 'https://dummyjson.com/products';

  getProducts() {
    return this.http.get<ProductsResponse>(this.baseUrl);
  }
}
```

`getProducts()` does **not** call `.subscribe()` itself — it hands back a "recipe" and the caller decides when to run it. `<ProductsResponse>` is a compile-time promise to TypeScript, not a runtime check — keep your interfaces accurate.



4 · WRITING DATA BACK

POST · PUT · DELETE, params & headers

```
addProduct(p: Partial<Product>) {
  return this.http.post<Product>(this.baseUrl + '/add', p);
}
updateProduct(id: number, changes: Partial<Product>) {
  return this.http.put<Product>(`${this.baseUrl}/${id}`, changes);
}
deleteProduct(id: number) {
  return this.http.delete(`${this.baseUrl}/${id}`);
}
```

Query params: build them with `HttpParams` instead of string concatenation — it encodes special characters safely. **Headers:** attach an auth token with `HttpHeaders`, e.g. `Authorization: Bearer <token>`.

5 · INTERCEPTORS

One place for logic that runs on every request

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');
  const cloned = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;
  return next(cloned);
};
```

Common uses: attaching auth tokens automatically, global error handling (auto-logout on 401), a global loading indicator, common headers — without repeating the logic in every `Service` method.

6 · BEST PRACTICES

COMMON MISTAKES

API calls directly inside components

Using **any** everywhere

No error handling on `subscribe()`

No loading state — UI looks frozen

Hardcoding URLs everywhere

BETTER ARCHITECTURE

Component → Service → HttpClient → API

Typed interfaces for every response

Centralized error handling + interceptors

Explicit loading / error / empty / success states

Base URLs in environment files